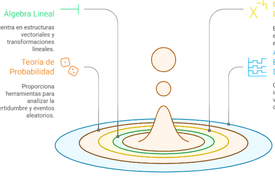
Unidad 3.1. Fundamentos de Machine Learning

Introducción al Machine Learning

El **Machine Learning (ML)** es un campo de la Inteligencia Artificial que permite a los sistemas aprender patrones a partir de datos sin ser programados explícitamente. Se basa en la idea de que las computadoras pueden **mejorar su desempeño en una tarea a medida que reciben más datos.**

Para comenzar a explorar el ML, primero profundizaremos algunos conceptos con Python:



Álgebra lineal con Python

Para manipular objetos (arreglos/arrays) multidimensionales, Python cuenta con la librería Numpy.

```python
import numpy as np

# Definir matrices A y B
A = np.array([[2, 4], [1, 3]])
B = np.array([[5, 1], [2, 6]])

# Suma de matrices
suma = A + B
print(f"Suma de matrices (A + B):\n{suma}")

# Resta de matrices
resta = A - B
print(f"\nResta de matrices (A - B):\n{resta}")
```

Suma y resta de matrices:

1. Verificar que las matrices A y B tengan el mismo tamaño $m \times n$.
2. Cada elemento de la matriz A se suma o resta con el elemento que ocupa la misma posición en la matriz B.
3. Se obtiene la matriz resultante C.

Ej: Suma de las matrices A y B.

$$A = \begin{pmatrix} 2 & 4 \\ 1 & 3 \end{pmatrix} B = \begin{pmatrix} 5 & 1 \\ 2 & 6 \end{pmatrix} C = \begin{pmatrix} 7 & 5 \\ 3 & 9 \end{pmatrix}$$

Producto Matricial con Python

1. Para multiplicar dos matrices A y B, el número de columnas de A debe ser igual al número de filas de B. Si A es de tamaño $m \times n$, entonces B debe ser de tamaño $n \times p$.
2. Se calcula cada elemento de C como el producto de una fila de A y una columna de B.
3. Se obtiene la matriz resultante C.

Ej.: Producto punto de las matrices A y B.

$$A = \begin{pmatrix} 2 & 4 \\ 1 & 3 \end{pmatrix} B = \begin{pmatrix} 5 & 1 \\ 2 & 6 \end{pmatrix}$$

$$C_{1,1} = (2 \cdot 5) + (4 \cdot 2) = 18$$
$$C_{1,2} = (2 \cdot 1) + (4 \cdot 6) = 26$$
$$C_{2,1} = (1 \cdot 5) + (3 \cdot 2) = 11$$
$$C_{2,2} = (1 \cdot 1) + (3 \cdot 6) = 19$$

$$C = \begin{pmatrix} 18 & 26 \\ 11 & 19 \end{pmatrix}$$

```python
# Producto matricial
producto_matrices = np.dot(A, B)
print(f"\nProducto matricial (A·B):\n {producto_matrices}")
```

Trasposición e Inversa de una Matriz

La transpuesta de una matriz A de tamaño $m \times n$ es una nueva matriz A^T de tamaño $n \times m$, donde las filas de A se convierten en las columnas de A^T y viceversa.

Ej.: Trasposta de A.

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix} A^T = \begin{pmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{pmatrix}$$

```python
# Transpuesta de A
transpuesta_A = A.T
print(f"\nTranspuesta de A:\n{transpuesta_A}")
```

⚠ Numpy cuenta con librerías dedicadas a distintas áreas de matemáticas y estadística. Un ejemplo el linalg para el Álgebra Lineal.

La inversa de una matriz cuadrada A de tamaño $n \times n$ es una matriz A^{-1} tal que:

$$A \cdot A^{-1} = A^{-1} \cdot A = I$$

Donde I es la matriz identidad de tamaño $n \times n$.

$$A = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \; det(A) = ad - bc$$

$$A^{-1} = \frac{1}{det(A)} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$$

```python
# Inversa de A (si es invertible)
if np.linalg.det(A) != 0:  # Verificamos que sea invertible
    inversa_A = np.linalg.inv(A)
    print(f"\nInversa de A:\n{inversa_A}")
else:
    print("\nLa matriz A no es invertible")
```

Cálculo Diferencial en el Machine Learning

Es fundamental en **Machine Learning**, ya que permite:

- Optimizar **funciones de costo** (minimizar errores en modelos).
- Actualizar **pesos en redes neuronales** mediante descenso del gradiente.
- Calcular **tasas de cambio** en modelos predictivos.

El **Gradiente Descendiente** es un algoritmo de optimización usado para minimizar funciones de costo en Machine Learning.

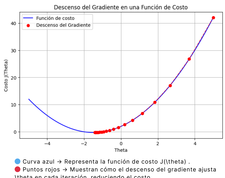
1. Se calcula la **derivada (gradiente)** de la función de costo.
2. Se mueven los parámetros en la **dirección opuesta** al gradiente.
3. Repetir hasta encontrar un **mínimo óptimo**.

Actualización del Gradiente:

$$\theta = \theta - \alpha \frac{dJ}{d\theta}$$

dónde,

- θ es el parámetro a optimizar.
- α es la tasa de aprendizaje.
- $J(\theta)$ es la función de costo a minimizar.



- Curva azul → Representa la función de costo J(θ)theta) .
- Puntos rojos → Muestran cómo el descenso del gradiente ajusta θtheta en cada iteración, reduciendo el costo.

Probabilidad y estadística en ML

El cálculo de **probabilidades y estadísticas** permite:

- **Modelar la incertidumbre** en datos reales.
- Tomar **decisiones óptimas** en algoritmos de clasificación y predicción.
- Evaluar y validar modelos con **métricas estadísticas**.

Las medidas de **Tendencia Central y Dispersión** son fundamentales para el análisis y resumen de los datos:

- **Media, Mediana y Moda** → Usadas para resumir datos.
- **Varianza y Desviación Estándar** → Miden la dispersión de los datos.

Distribuciones de Probabilidad

- **Normal (Gaussiana):** Modela datos de tendencia central
- **Bernoulli o Binomial:** Eventos de éxito o fracaso.
- **Poisson:** Eventos entre intervalos de tiempo.

Análisis Exploratorio de Datos - EDA

Visualización de datos e identificación de outliers y patrones.

Modelos Probabilísticos

- **Redes Bayesianas:** Modelan relaciones de causa y efecto con probabilidades condicionales.
- **HMM (Modelos Ocultos de Markov):** Aplicados en procesamiento de lenguaje natural y detección de fraudes.

Aplicaciones del Teorema de Bayes

El Clasificador ***Naive Bayes*** asume que todas las características son independientes entre sí. Se usa ampliamente en:

- Filtrado de spam en correos electrónicos.
- Clasificación de textos y análisis de sentimientos.
- Diagnóstico médico basado en síntomas.

Fórmula aplicada a clasificación:

$$P(y|X) = \frac{P(X|y)P(y)}{P(X)}$$

donde:

$P(y|X)$ → Probabilidad de una clase y dado los datos X.

$P(X|y)$ → Probabilidad de observar los datos X si y es verdadera.

$P(y)$ → Probabilidad de y antes de observar X (probabilidad previa).

$P(X)$ → Probabilidad total de los datos.

Análisis Exploratorio de Datos - EDA

Resumen Estadístico
Calcula la media, mediana, desviación estándar, mínimo y máximo.

Visualización de Datos
Crea histogramas, boxplots y gráficos de dispersión.

Correlaciones
Identifica relaciones entre variables para seleccionar características útiles.

Preprocesamiento de datos

Limpieza y Tratamiento de Datos

Es el proceso que detectar, corregir o eliminar errores e inconsistencias en los datos con el fin de mejorar la calidad de la información.

```
import pandas as pd

# Crear DataFrame de ejemplo con valores faltantes
df = pd.DataFrame({
    'Edad': [25, 30, None, 40, None],
    'Salario': [50000, None, 60000, 65000, None],
    'Ciudad': ['Madrid', 'Barcelona', None, 'Valencia', 'Sevilla']
})

# Eliminar filas con valores faltantes
df_sin_filas_nulas = df.dropna()

# Eliminar columnas con valores faltantes
df_sin_columnas_nulas = df.dropna(axis=1)

print(df_sin_filas_nulas)
print(df_sin_columnas_nulas)
```

Eliminación

Consiste en la remoción de filas o columnas con valores faltantes.

- Se usa cuando el **porcentaje de valores faltantes** es alto (> 30 - 40% de los datos).
- Solo si la información faltante es **irrelevante** para el análisis.
- Puede **perderse información** valiosa.

Imputación

Remplazar los valores faltantes con valores estimados.

- Se usa cuando la **cantidad de datos faltantes** es bajo (< 30%).
- Es preferible cuando la **eliminación** **recurrida** demasiado el dataset.
- Hay **diferentes técnicas** de estimación:
  - **Media/mediana:** Para datos numéricos.
  - **Moda:** Para datos categóricos.
  - **Modelos predictivos** en función de otras variables.

```
import pandas as pd
import numpy as np
from sklearn.impute import SimpleImputer

# Imputar con la media para datos numéricos
imputer = SimpleImputer(strategy='mean')
df['Edad', 'Salario']] = imputer_media.fit_transform(df['Edad', 'Salario'])

# Rellenar moda para ciudad (categórico)
df['Ciudad'] = df['Ciudad'].replace({None: np.nan})

# Imputar con la moda para datos categóricos
imputer_moda = SimpleImputer(strategy='most_frequent')
df[['Ciudad']] = imputer_moda.fit_transform(df[['Ciudad']])
```

Detección de Outliers

Los **outliers** son valores extremos que se alejan significativamente del resto de los datos y pueden distorsionar modelos de Machine Learning.

Método estadístico: Z-score

El Z-score mide cuántas desviaciones estándar está un valor de la media.

- Valores con $|Z|>3$ suelen considerarse **outliers**.
- Se usa en datos con distribución normal.

```
import numpy as np
import pandas as pd
from scipy.stats import zscore

# Fijar semilla para reproducibilidad
np.random.seed(42)

# Crear 99 valores con distribución normal (media 50, std 5)
normal_values = np.random.normal(50, 5, 99)

# Añadir un outlier extremo (por ejemplo, 100)
valores = np.append(normal_values, 100)

# Crear DataFrame
df = pd.DataFrame({'Valor': valores})

# Calcular el Z-score
df['Z_score'] = zscore(df['Valor'])

# Filtrar filas que son outliers (Z > 3 o Z < -3)
outliers = df[abs(df['Z_score'])> 3]

print("DataFrame completo:\n", df)
print("\nOutliers detectados:\n", outliers)
```

Método gráfico: Boxplot

El **Boxplot** es una herramienta visual que usa los cuartiles para identificar valores atípicos:

Regla del Rango Intercuartílico:

$$IQR = Q3 - Q1$$

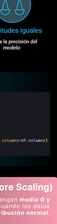
Un **outlier** es un valor fuera del rango:

$$[Q1 - 1.5 \times IQR, Q3 + 1.5 \times IQR]$$

Método Gráfico

```
import matplotlib.pyplot as plt
import seaborn as sns

# Visualización de outliers con Boxplot
plt.figure(figsize=(5,4))
sns.boxplot(data=df['Valor'])
plt.title("Detección de Outliers con Boxplot")
plt.show()
```



Tratamiento de Outliers

Eliminación de *Outliers*

Si el número de **outliers** es pequeño y no aporta información útil, se pueden eliminar.

```
# Filtrar valores dentro del rango aceptable usando IQR
Q1, Q3 = np.percentile(df['Valor'], [25, 75])
IQR = Q3 - Q1
limite_inferior = Q1 - 1.5 * IQR
limite_superior = Q3 + 1.5 * IQR

df_sin_outliers = df[
    (df['Valor'] > limite_inferior) &
    (df['Valor'] < limite_superior)
]

print(df_sin_outliers)
```

Transformación de datos

Si los **outliers** son parte natural de los datos, se pueden aplicar:

- **Escalado Logarítmico:** Reduce el impacto de valores extremos.

```
df['Valor_transformado'] = np.log(df['Valor'] + 1)
```

- **Winsorización:** Reemplaza *outliers* con percentiles extremos.

```
from scipy.stats.mstats import winsorize
df['Valor_winsorizado'] = winsorize(df['Valor'], limits=[0.05, 0.05])
```

Escalado y Normalización de Datos

¿Por qué es necesario escalar los datos?

- Suponga un dataset que contenga medidas como el **peso (kg) y la altura (cm)**.
- La altura se encuentra en rangos de valores entre **100 y 200**, mientras que el peso varía entre **50 y 100**.
- La diferencia en las magnitudes puede provocar que algunas características predominen sobre otras en modelos como la **Regresión Lineal, k-NN, Redes Neuronales y SVM**.

¿Cómo manejar las diferencias de magnitud en los datos?

Normalización (Min-Máx Scaling)

Convierte los valores al rango **[0,1] o [-1,1]**, asegurando que todas las características tengan la **misma escala**.

Se calcula como:

$$X' = \frac{X - X_{min}}{X_{max} - X_{min}}$$

```
from sklearn.preprocessing import MinMaxScaler

# Crear DataFrame ejemplo
df = pd.DataFrame({'Altura': [150, 160, 170, 180, 190],
                   'Peso': [50, 60, 70, 80, 100]})

# Aplicar normalización (Min-Max Scaling)
scaler = MinMaxScaler()
df_normalizado = pd.DataFrame(scaler.fit_transform(df), columns=df.columns)

print("Datos normalizados\n", df_normalizado)
```

Estandarización (Z-Score Scaling)

Ajusta los datos para que tengan **media 0 y desviación estándar 1**. Es útil cuando los datos siguen una **distribución normal**.

Se calcula como:

$$X' = \frac{X - \mu}{\sigma}$$

```
from sklearn.preprocessing import StandardScaler

# Crear DataFrame ejemplo
df = pd.DataFrame({
    'Altura': np.random.normal(loc=200, scale=5, size=100),
    'Peso': np.random.normal(loc=150, scale=1, size=100)
})

# Aplicar estandarización (Z-score)
scaler_std = StandardScaler()
df_estandarizado = pd.DataFrame(scaler_std.fit_transform(df),
                                columns=df.columns)

print("Datos estandarizados\n", df_estandarizado)
print("Media después de estandarización:\n", df_estandarizado.mean())
print("Desviación estándar después de estandarización:\n",
print("DESPUÉS DE ESTANDAR: \n",round(df_estandarizado.std(), 2))
```

Codificación de Variables Categóricas

Label Encoding

Asigna un número entero a cada categoría

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder

# Crear DataFrame de ejemplo
df = pd.DataFrame({'Color': ['Rojo', 'Azul',
                             'Verde', 'Azul',
                             'Rojo']})

# Aplicar Label Encoding
encoder = LabelEncoder()
df['Color_codificado'] = encoder.fit_transform(df['Color'])

print(df)
```

Color	Color Codificado
Rojo	2
Azul	0
Verde	1
Azul	0
Rojo	2

¿Qué método usar para la codificación de variables categóricas?

Label Encoding
Simple y eficiente para puede introducir un orden artificial.

One-Hot Encoding
Previene el orden artificial, pero puede aumentar la dimensionalidad.

One-hot Encoding

Crea nuevas **columnas binarias (0 o 1)** para cada categoría.

Color	Color_Azul	Color_Rojo	Color_Verde
Rojo	0	1	0
Azul	1	0	0
Verde	0	0	1
Azul	1	0	0
Rojo	0	1	0

```
from sklearn.preprocessing import OneHotEncoder

# Aplicar One-Hot Encoding
encoder = OneHotEncoder(sparse_output=False)
encoded = encoder.fit_transform[df['Color']]

# Convertir a DataFrame con nombres de columnas
df_encoded = pd.DataFrame(encoded, columns=encoder.get_feature_names_out(['Color']))

# Eliminar la columna de la codificación anterior
df = df.drop('Color')+df_codificado', axis=1)

# Concatenar con el dataframe original
df_final = pd.concat([df, df_encoded], axis=1)

print(df_final)
```

¿Cuándo es mejor usar cada método?

Situación	Label Encoding	One-Hot Encoding
Variables ordenadas (Alto, medio, bajo)	✓Sí	✗No
Variables sin jerarquía (Colores)	✗No	✓Sí
Muchas categorías únicas (ciudades)	✓Sí	✗No (puede generar muchas columnas)

Reducción de Dimensionalidad

Análisis de Componentes Principales

En datasets con muchas características (**alta dimensionalidad**), algunos atributos pueden ser redundantes o irrelevantes.

Técnicas como el **Análisis de Componentes Principales (PCA)** ayuda a reducir la cantidad de variables sin perder demasiada información importante.

- Es una técnica que transforma las variables originales en un nuevo conjunto de variables llamadas **componentes principales**.
- Retiene la **mayor cantidad de varianza** posible en los datos.
- Reduce el **tiempo de entrenamiento** y mejora la eficiencia del modelo.

Unidad 3.2. Implementación de modelos de Machine Learning

Aprendizaje Supervisado

En un tipo de aprendizaje automático donde el modelo **aprende a partir de datos etiquetados**, es decir, conoce la entrada (X) y la salida esperada (y).

Se divide en dos tipos principales:

- **Regresión:** Predicción de valores continuos.
- **Clasificación:** Predicción de clases o categorías.

Aprendizaje No Supervisado

En un enfoque de Machine Learning en el que los modelos aprenden **por sí mismos sin etiquetas o valores objetivos**.

- El objetivo es encontrar **estructuras, patrones o agrupaciones en los datos**. Se usa cuando no se conocen las etiquetas o cuando **se desconoce información latente**.

Regresión vs. Clasificación

Características	Regresión	Clasificación
Tipo de salida	Valor continuo (número real)	Clase o categoría (etiqueta discreta)
Ejemplos típicos	• Predicción de precios (casas, acciones, etc.) • Predicción de temperatura o demanda energética • Predicción de notas o tamaño	• Diagnóstico médico (enfermo / sano) • Reconocimiento de imágenes (gato / perro) • Detección de fraude (fraude / legítimo)
Objetivo del modelo	Aproximar una función continua $f: X \to \mathbb{R}$	Aprender a asignar etiquetas $f: X \to \{0, 1, \ldots\}$
Algoritmos comunes	• Regresión Lineal • Regresión Logística • Lasso, Ridge	• k-NN (k-vecinos más cercanos) • Naive Bayes • Random Forest, SVM
Métricas de evaluación	• MAE (error absoluto medio) • MSE (error cuadrático medio) • R^2 (coeficiente de determinación)	• Precisión, Recall, F1-score • Matriz de confusión • Curvas ROC y AUC

Regresión con Aprendizaje Supervisado

Regresión Lineal Simple

La **Regresión Lineal Simple** es el modelo más básico de regresión. Busca encontrar la **línea recta** que mejor se ajusta a un conjunto de datos con **una sola variable independiente** (X) y **una variable dependiente** (Y).

- Encuentra los valores de β_0 y β_1 que **minimizan el error** entre nuestras los valores predichos y los observados.
- La línea resultante representa la **relación lineal más ajustada** posible.

Se representa por:

$$Y = \beta_0 + \beta_1 X + \epsilon$$

- y: variable dependiente (lo que queremos predecir)
- x: variable independiente (lo que sabemos)
- β_0: intercepto
- β_1: pendiente (coeficiente de regresión)
- ϵ: error (residuo)

Suponiendo los datos:

Tamaño (m²)	Precio (USD)
50	100
80	160
120	200

Evaluación de los modelos de Regresión

Después de entrenar un modelo de regresión, es fundamental evaluar su desempeño para saber qué tan bien predice sobre datos nuevos.

Métrica	Fórmula	Interpretación	Significado
MAE (Mean Absolute Error)	$MAE = \frac{1}{n}\sum_{i=1}^{n} \lvert y_i - \hat{y}_i \rvert$	Mide cuánto se equivoca, en promedio y en unidades originales.	Cuanto menor sea, mejor.
MSE (Mean Squared Error)	$MSE = \frac{1}{n}\sum_{i=1}^{n} (y_i - \hat{y}_i)^2$	Penaliza más los errores grandes.	Útil para evitar errores significativos.
RMSE (Raíz del MSE)	$\sqrt{MSE}$	Escala comparable a la variable original.	Útil para comparar modelos.
R^2 (Coeficiente de determinación)	$1 - \frac{SS_{res}}{SS_{tot}}$	Cuánta varianza explica el modelo. Valor entre 0 y 1.	1 = ajuste perfecto, 0 = el modelo no explica nada.

Ejemplo de Regresión: California Housing Dataset

El dataset California Housing fue creado a partir del **censo de 1990 en California**, EE.UU. Cada fila representa **un valor medio** de las viviendas en distintos bloques geográficos, a partir de datos socioeconómicos y demográficos.

Características:

- **Observaciones:** 20,640 muestras
- **Variables independientes:** 8 variables numéricas
- **Variable objetivo (y):** **MedHouseVal**: valor medio de la vivienda (en centenas de miles de dólares)

Algoritmos/Modelos de Regresión

Modelo	Tipo	Penalización / Regularización	Ventajas	Limitaciones
Regression Lineal Multiple	Lineal	Sin penalización	Sencillo, interpretabilidad alta	Sensible a outliers, asume linealidad
Ridge Regression	Lineal	L2 (penaliza coeficientes grandes)	Controla multicolinealidad, evita sobreajuste	No realiza selección de variables
Lasso Regression	Lineal	L1 (reduce coeficientes a cero)	Realiza selección de variables automáticamente	Puede eliminar variables relevantes si están correlacionadas
Árboles de Regresión	No lineal	X Ninguna	Maneja no linealidad y interacciones, interpretabilidad alta	Propenso a sobreajuste (a no ser de poder)
Random Forest Regressor	No lineal	X Ensamble de árboles	Reducción de varianza, robusto a outliers y ruido	Menos interpretable, requiere más recursos computacionales
XGBoost Regressor	No lineal	X Gradient boosting	Precisión alta, manejo eficiente de missing values y overfitting	Mayor complejidad, tuning sensible
SVR (Support Vector Regression)	Lineal/No lineal	X Margen + kernel	Preciso en espacios complejos, robusto ante valores no lineales	Lento en grandes datasets, requiere tuning fino de hiperparámetros

¿Cómo elegir un modelo?

No existe una regla estricta al respecto, pero algunas recomendaciones sobre los modelos más populares son:

- Datos pequeños + pocas variables relevantes → **Lasso** o **Ridge**
- Datos con relaciones no lineales → Polinómica, Árboles, SVR
- Necesidad de robustez y precisión → Random Forest o XGBoost
- Mayor interpretabilidad → Lineal o Lasso

Clasificación con Aprendizaje Supervisado

La clasificación es una tarea de aprendizaje supervisado cuyo objetivo es **predecir la clase o categoría** a la que pertenece una observación, a partir de un conjunto de variables predictoras.

- La variable objetivo es **discreta (categórica)**.
- El modelo aprende de **datos etiquetados**, es decir, ejemplos donde ya se conoce la clase correcta.

Tipos de Clasificación

Tipo	Ejemplo de Categorías
Binaria	Enfermo vs. Sano
Multiclase	Categorización de flores (setosa, versicolor, virginica)
Multietiqueta	Una imagen puede tener varias etiquetas (ej: gato y ratón)

Ejemplos de uso

Caso	Entrada (X)	Clase esperada (y)
Diagnóstico médico	Síntomas del paciente	Enfermo / Sano
Clasificación de correos	Texto del mensaje	Spam / No spam
Detección de fraudes	Datos de transacciones	Fraude / No fraude
Reconocimiento de imágenes	Píxeles de la imagen	Gato / Perro / Ave / ...

Algoritmos de Clasificación

Algoritmo	Tipo de modelo	Ventajas	Limitaciones
k-NN	Basado en instancias	• Simple, no requiere entrenamiento • Funciona bien en datos bien escalados	• Lento en predicción • Sensible a la escala y al ruido
Árboles de Decisión	Modelo jerárquico	• Fácil de interpretar • Maneja variables numéricas y categóricas	• Propenso a sobreajuste en solo
Random Forest	Ensamble (bagging)	• Robusto y preciso • Reduce el overfitting	• Menos interpretable • Mayor costo computacional
SVM	Frontera óptima	• Funciona bien en espacios de alta dimensión • Efectivo en clases separables	• Difícil de escalar a grandes datasets
Naive Bayes	Probabilístico	• Rápido y eficiente • Robusto con datos categóricos	• Supone independencia entre variables (poco realista en muchos casos)
Regresión Logística	Modelo lineal	• Interpretable • Rápido de entrenar	• Solo modela relaciones lineales entre variables y clases

Casos de uso comunes

Condiciones	Algoritmo recomendado
Dataset pequeño y sin muchas reglas	Naive Bayes o Regresión Logística
Alta dimensionalidad (muchas variables)	SVM o Random Forest
Interpretabilidad es clave	Árboles de Decisión o Regresión Logística
Clases bien separadas (geométricamente)	SVM o k-NN
Datos mixtos (categóricos + numéricos)	Árboles o Random Forest

Métricas de Evaluación en la Clasificación

Precisión (Accuracy)

- La variable objetivo es **discreta (categórica)**.
- El modelo aprende de **datos etiquetados**, es decir, ejemplos donde ya se conoce la clase correcta.

$$(TP + TN)/Total)$$

Recall (Sensibilidad)

- Mide cuántos del total de positivos predijo correctamente.
- Útil en diagnósticos médicos al minimizar los FN.

$$(TP/(TP + FN))$$

F1-Score

- Busca el equilibrio con la media armónica entre la Precisión y el Recall.
- Se usa con clases desbalanceadas.

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

Curva ROC (Receiver Operating Characteristic) y AUC (Area Under the Curve)

- El gráfico muestra la **Tasa de Verdaderos Positivos (TVP) vs Tasa de Falsos Positivos (FPR)** en distintos umbrales.
- El AUC es el **área debajo de la curva del ROC** que resume el rendimiento del modelo en todos los umbrales posibles. Su valor va entre cero y 1, siendo el modelo tiene **mejor capacidad de discriminación** entre más se aproxime a 1.

k-Nearest Neighbors (k-NN)

k-NN (k-Nearest Neighbors) es un algoritmo de clasificación **basado en la proximidad**. Para clasificar una nueva muestra, el modelo:

- Busca los k vecinos más cercanos en el conjunto de entrenamiento.
- Asigna la categoría (clasificación) o promedio (regresión) según las clases de esos vecinos.
- Ideal para problemas donde la geometría del espacio es relevante.

Para clasificar un punto nuevo (x,y), busca sus k vecinos más cercanos según una métrica (por defecto, distancia Euclidiana):

$$dist(x, x_i) = \sqrt{\sum_{j=1}^{d} (x_j - x_{ij})^2}$$

Se usa con clases **balanceadas**.

Parámetros Clave en k-NN

Parámetro	Descripción
n_neighbors	Número de vecinos a considerar. Generalmente impar.
metric	Métrica de distancia (euclidean, manhattan, minkowski)
weights	Puede ponderar por distancia (uniform o distance)

Ventajas

- Fácil de entender e implementar.
- Funciona bien con datos bien separados.

Desventajas

- Lento en predicción si hay muchos datos.
- Sensible a escala, outliers y datos ruidosos.

Visualización de las clases predichas

- Las regiones de color muestran cómo el modelo asignaría una clase en función de la cercanía a los datos reales.
- Los puntos negros representan los datos de entrenamiento.
- La forma de las regiones cambia al variar el valor de k.

Árboles de Decisión (Desicion Trees)

Un **Arbol de Decisión** es un modelo predictivo que **clasifica o predice valores** mediante **reglas jerárquicas** en ramas, siendo **preguntas binarias** del tipo:

¿"Ingreso > 50k"?
¿"Longitud de pétalo ≤ 2.5"?

Cada nodo representa una **decisión basada en una variable**, y cada hoja representa una **clase final**.

- Funciona como una estructura tipo **"si... entonces..."** (if-else).
- Puede usarse tanto para **clasificación** como para **regresión**.

¿Cómo funcionan los Árboles de Decisión?

- Selecciona la **mejor variable** y **umbral** para dividir los datos.
- Repite este proceso **recursivamente en cada subconjunto**.
- Se detiene al llegar a **condiciones de parada**:
  - **Profundidad máxima** alcanzada.
  - Todas las muestras tienen la **misma clase**.
  - Número **mínimo de muestras** en una hoja

Parámetros Clave en los Árboles de Decisión

Parámetro	Descripción
max_depth	Profundidad máxima del árbol
min_samples_split	Mínimo de muestras para dividir un nodo
criterion	Métrica: "gini" o "entropy"
max_features	Número de variables consideradas en cada división

Aprendizaje No Supervisado

En un enfoque de Machine Learning en el que los modelos aprenden **sin utilizar etiquetas ni valores objetivos**.

- El objetivo es encontrar **estructura, patrones o agrupaciones** en los datos.
- Es ideal para una variable explícita, sino que **se desconoce información latente**.

Tipos de Aprendizaje No Supervisado:

Categoría	Descripción
Clustering	Agrupar ejemplos similares según sus características.
Reducción de Dimensionalidad	Reducir el número de variables sin perder información relevante.

Aplicaciones del Clustering

Caso	Uso del Clustering
Segmentación de clientes	Agrupar consumidores por comportamiento
Análisis de imágenes	Agrupar pixeles o regiones similares
Detección de anomalías	Identificar valores atípicos como "ruido"
Bioinformática	Agrupar genes con expresión similar
Agrupación de documentos	Clasificación de textos sin etiquetas previas

Clustering

El clustering es una técnica que busca **agrupar instancias similares** en conjuntos llamados **clusters**, sin usar **etiquetas previas**.

Objetivo:
Descubrir **estructura o patrones ocultos** en los datos, asignando a cada observación **un grupo basado** en su proximidad o similitud.

¿Cómo funciona el Clustering?

- Calcula una **medida de distancia o similitud** (euclidiana, Manhattan, coseno, etc.).
- Agrupa los puntos que **están cerca entre sí** en el espacio de características.

¿Cómo se combinan el Clustering con la Reducción de Dimensionalidad?

El mayo consiste aplicar reducción de dimensionalidad (**PCA, t-SNE**) antes de hacer clustering para:

- **Visualizar** los **agrupados**.
- **Visualizar los clusters** en un plano 2D.

Reducción de Dimensionalidad

Proceso de transformar un **conjunto de datos de alta dimensión** en otro de menor dimensión, manteniendo su **estructura esencial**.

Se usa para:

- **Visualizar** datos en 2D o 3D.
- **Acelerar algoritmos** que escalan bien con muchas variables.
- **Eliminar ruido** o redundancia en las variables.

Clustering con K-Means

K-Means es un algoritmo de agrupamiento que busca **dividir los datos en k grupos o clusters**, de modo que los puntos dentro de cada cluster sean **lo más similares posible entre sí** y diferentes a los de otros clusters.

Funcionamiento de K-Means:

1. **Inicialización:** Elige aleatoriamente k puntos como **centroides iniciales** (centros de cada cluster).
2. **Asignación:** Asigna cada dato al **centroide más cercano** (según una métrica, generalmente distancia).
3. **Actualización:** Recalcula los centroides como la **media** de los puntos asignados a cada cluster.
4. **Repetición:** Repite pasos 2 y 3 hasta que los centroides **dejen de cambiar** (o se alcance un número máximo de iteraciones).

Evaluación del Clustering

Como no hay etiquetas, usamos métricas del mismo modelo:

Métrica	Descripción
Inercia (SSE)	Suma de distancias cuadradas al centroide
Silhouette Score	Mide la calidad de la separación de clusters
Davies-Bouldin	Compactación, mayor separación y compactación

Manipular el comportamiento

- Cambiar **k** a **2** o **4** para ver cómo afecta el número de clusters.
- Aumentar cluster_std en make_blobs para ver cómo responde a datos **más dispersos**.

Ejemplo K-Means

- Se generan datos con **3 grupos naturales** (usando make_blobs).
- k-Means **descubre** esos los clusters automáticamente.
- Se grafican los centroides en negro.
- Se calcula el Silhouette Score (qué tan bien la calidad del agrupamiento).
  - Rango de -1 (malo) a 1 (excelente).
  - Un valor > 0.5 es generalmente considerado bueno.

Siguiente →

Modelos Avanzados de Machine Learning

Esta sección profundiza en algoritmos más potentes y complejos que los modelos base, útiles cuando:

✅ Se trabaja con grandes volúmenes de datos
✅ Se necesita mejor rendimiento predictivo
✅ Se desea robustez frente a ruido o irrelevancia en los datos

Modelos de Ensamble

Un modelo de ensamble **combina múltiples modelos** base (por lo general, débiles o moderados) para formar un modelo **más robusto y preciso**.

Ventajas

- Reducen el **riesgo de sobreajuste**
- Mejoran la **generalización a nuevos datos**
- Son **más estables** frente al ruido o cambios menores en los datos
- A menudo **superan modelos individuales** en competencia

Estrategias comunes de Ensamble

Tipo	Ejemplo	¿Cómo funciona?
🧺 Bagging	Random Forest	Entrena modelos en subconjuntos aleatorios del dataset y los promedia o vota
📈 Boosting	XGBoost, AdaBoost	Entrena modelos secuencialmente, cada uno corrige los errores del anterior
🧱 Stacking	Modelos apilados	Combina varios modelos usando otro modelo (meta-modelo) para tomar la decisión final

*"Un modelo de ensamble **no intenta encontrar un modelo perfecto**, sino combinar varios buenos modelos que se **equivoquen de forma diferente**."*

Random Forest

Random Forest es un algoritmo de ensamble basado en **bagging** que construye muchos **árboles de decisión independientes** y los combina para hacer una predicción **más precisa y estable**.

Funcionamiento

1. Toma **subconjuntos aleatorios** del dataset (con reemplazo).
2. Construye **un árbol de decisión** para cada subconjunto.
3. En cada nodo, **elige la mejor división** entre un subconjunto aleatorio de características.
4. **Combina** las predicciones:
   - **Clasificación** → votación mayoritaria
   - **Regresión** → promedio

✅Ventajas

- Reduce el **sobreajuste** respecto a un solo árbol.
- Maneja bien **muchas variables y datos ruidosos**.
- Proporciona **importancia de características**.
- Funciona bien sin necesidad de **ajuste fino de hiperparámetros**.

⚠️ Limitaciones

- Puede ser **menos interpretable** que un árbol simple.
- Más **costoso computacionalmente** (muchos árboles).

*"Random Forest es como preguntar a **muchos expertos independientes** y luego seguir el consenso".*

XGBoost: Extreme Gradient Boosting

Es un algoritmo de Boosting altamente eficiente que crea un modelo fuerte al **entrenar secuencialmente** modelos débiles (como árboles pequeños), donde cada nuevo modelo **corrige los errores del anterior**.

- Basado en la técnica de **Gradient Boosting**
- Ampliamente utilizado por su **rendimiento y velocidad optimizados**
- Ganador frecuente en **competencias como Kaggle**

Funcionamiento

- Cuando buscas alta precisión y generalización
- En datasets con ruido y correlaciones
- Permite el ajuste fino de hiperparámetros
- Ideal para datasets estructurados/tabulares

Características

- Manejo de valores faltantes automáticamente
- Regularización incorporada (L1 y L2)
- Control de sobreajuste (con max_depth, eta, subsample)
- Soporta paralelización y GPU

Parámetros Clave XGBoost

Parámetro	Descripción
n_estimators	Número de árboles (boosting rounds)
max_depth	Profundidad máxima de cada árbol
learning_rate (eta)	Cuánto corrige cada nuevo árbol
subsample	Proporción de datos usada por árbol
colsample_bytree	Proporción de variables por árbol

Diferencias entre XGBoost y Random Forest

Característica	Random Forest (Bagging)	XGBoost (Boosting)
Entrenamiento	Paralelo, árboles independientes	Secuencial, árboles dependientes
Estrategia	Reduce varianza	Reduce sesgo
Combinación	Votación o promedio	Suma ponderada de predicciones
Regularización	No incluida por defecto	Regularización L1 y L2
Rendimiento	Muy bueno	Excelente (mejor ajuste fino)

AutoML (Automated Machine Learning)

1. Explora diferentes **pipelines** de preprocesamiento + modelos
2. Evalúa modelos con **validación cruzada**
3. **Optimiza** con algoritmos genéticos, búsqueda bayesiana o aleatoria
4. Devuelve el **mejor modelo** con sus métricas y configuración óptima

*"AutoML **no reemplaza** al científico de datos, pero libera tiempo de **trabajo técnico repetitivo** para enfocarse en lo **estratégico**."*

TPOT (Tree-based Pipeline Optimization Tool):	Auto-sklearn:
• Usa **algoritmos genéticos** para encontrar el mejor pipeline de ML. • Automatiza: preprocesamiento + modelo + hiperparámetros. • 100% basado en Scikit-learn. • Muy fácil de integrar en proyectos existentes.	• Usa **optimización bayesiana** para explorar y ajustar modelos. • Integra múltiples algoritmos (SVM, árboles, etc.) • Requiere más configuración de entorno (compilador C, etc.). • Más potente para tareas complejas, pero menos transparente.

PyCaret como AutoML

- Detecta **tipo de variables, escala, codifica y limpia automáticamente**
- Prueba **+10 modelos** de clasificación automáticamente
- Permite **visualizar y comparar** resultados (AUC, precisión, matriz de confusión...)

Característica	PyCaret	Auto-sklearn / TPOT
Enfoque	Bajo código, flujo guiado	Auto-optimización agresiva
Transparencia	Alta (puedes ver cada paso)	Menor (caja negra más cerrada)
Curva de aprendizaje	Muy baja (ideal para enseñar)	Requiere algo más de control
Flexibilidad	Alta (compatible con scikit-learn y modelos personalizados)	Alta también, pero más técnica

Optimización de hiperparámetros en ML

Los hiperparámetros son **configuraciones externas al modelo** que afectan su rendimiento pero **no se aprenden directamente** del entrenamiento.

Ejemplos comunes de hiperparámetros

- ***n_neighbors*** en k-NN
- ***max_depth*** en árboles de decisión
- ***C*** y ***gamma*** en SVM
- ***learning_rate*** en XGBoost

Beneficios de la optimización

✅ Mejorar la **generalización** del modelo
✅ Reducir el **error de validación**
✅ Para evitar tanto **overfitting como underfitting**

Técnicas comunes de búsqueda de hiperparámetros

Método	¿Cómo funciona?
Grid Search	Prueba todas las combinaciones posibles de hiperparámetros (exhaustivo)
Random Search	Prueba un subconjunto aleatorio de combinaciones (más rápido)
Bayesian Search	Usa resultados anteriores para elegir la siguiente mejor combinación a probar